

STM32F103 Custom Development Board KiCad PCB Design

Prepared by: Sbaz Ali Khan.

Table of Contents

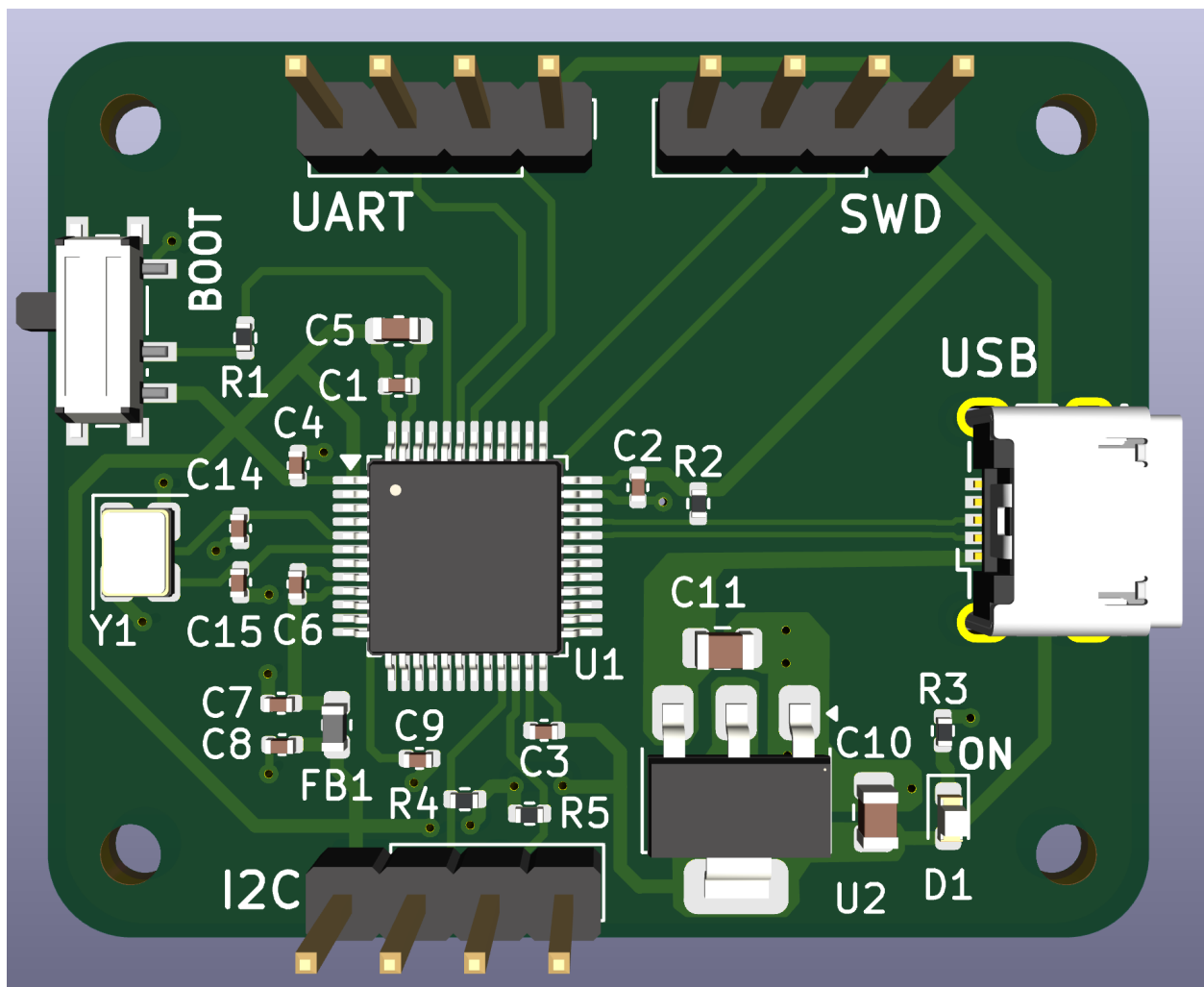
<i>STM32F103 Custom Development Board KiCad PCB Design</i>	<i>1</i>
1. Project Overview	7
2. Main Components.....	7
Microcontroller	7
Voltage Regulator.....	8
USB Connector	8
External Crystal	8
Communication Headers	8
Other Components	9
3. Basic Power System.....	9
4. Voltage Regulation.....	10
5. Decoupling Capacitors.....	11
5.1 Small Decoupling Capacitors.....	11
6. Bulk Decoupling Capacitor	12
7. Coupling vs Decoupling Capacitors	12
Decoupling Capacitor	12
Coupling Capacitor	12
8. Capacitor Behaviour During Power-Up.....	13
9. Digital and Analog STM32 Power	13
VDD	14
VSS	14
VDDA.....	14
VSSA	14
10. VDDA Filtering	14
11. Ferrite Bead	15
12. PWR_FLAG.....	16

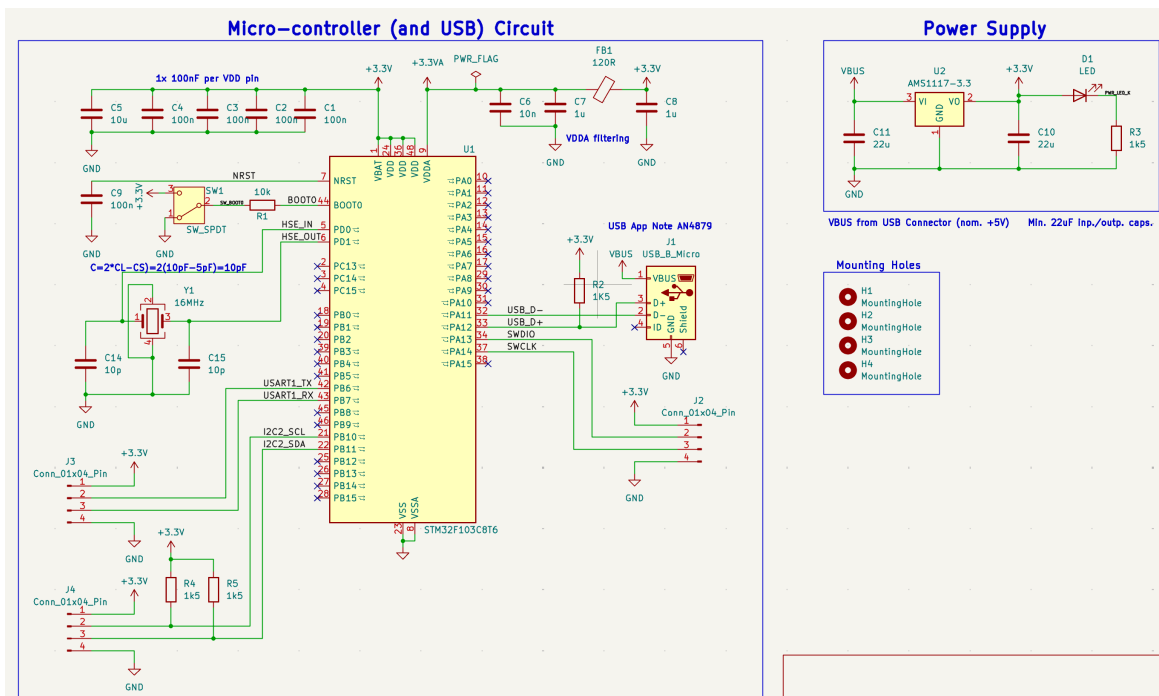
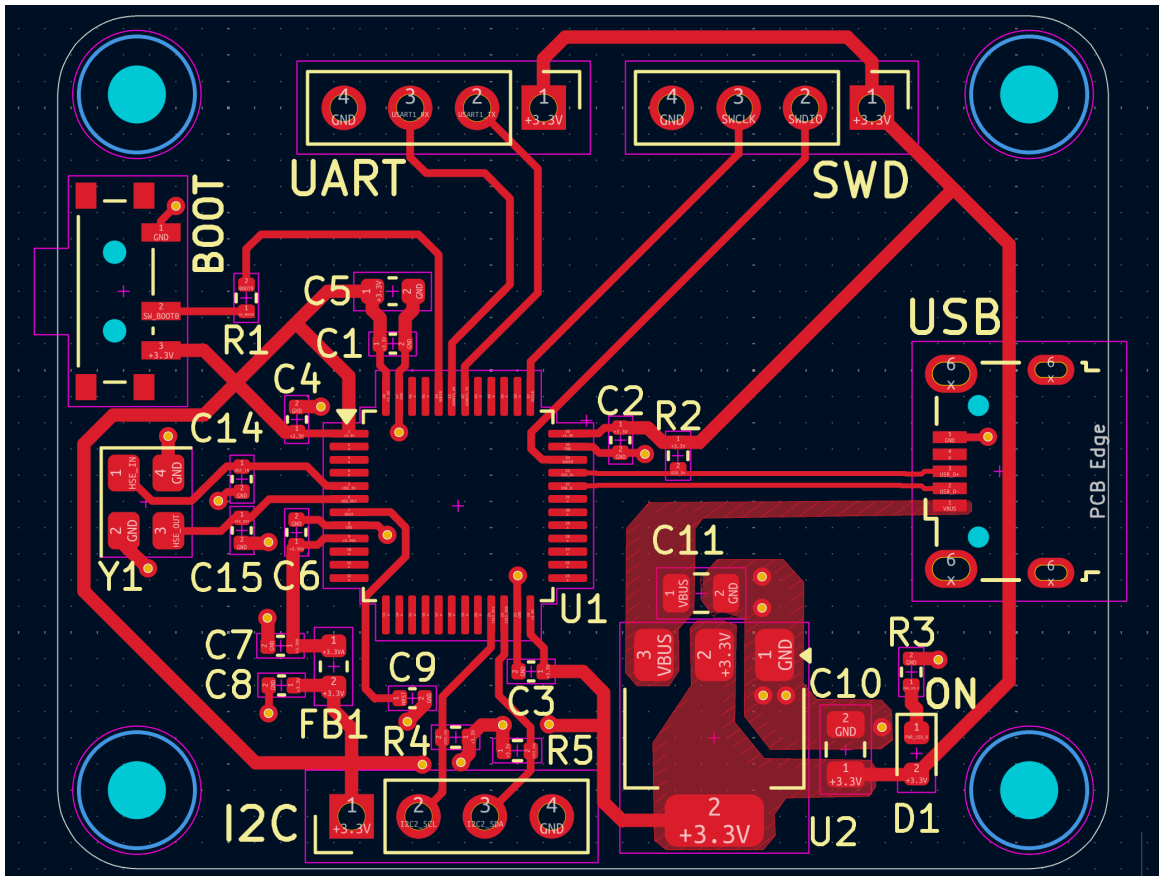
13. Reset Circuit — NRST	16
14. BOOT0.....	17
BOOT0 LOW.....	17
BOOT0 HIGH.....	17
15. BOOT0 Switch.....	18
16. External Crystal Oscillator.....	18
17. Crystal Load Capacitors	19
18. USB Interface.....	20
19. USB Differential Pair	21
20. USB D+ Pull-Up Resistor	22
21. USB ID Pin.....	22
22. USB Shield.....	23
23. Serial Wire Debug.....	23
SWDIO	23
SWCLK.....	23
24. What SWD Is Used For	24
25. USART / UART Interface.....	24
TX.....	24
RX.....	24
26. I²C Interface.....	26
SCL	26
SDA.....	26
27. Why I²C Requires Pull-Up Resistors	26
28. Why UART Does Not Need These Pull-Ups	27
29. STM32CubeIDE / CubeMX	27
30. Schematic Capture in KiCad	28
31. Symbols, Values and Footprints.....	29
Symbol.....	29
Value	29
Footprint.....	30
32. Selected Footprints	30
STM32.....	30
AMS1117	30

Small capacitors and resistors	30
10 μ F capacitor	30
22 μ F capacitors	31
LED.....	31
Ferrite bead	31
Crystal.....	31
Headers	31
Mounting holes.....	31
33. Component Package Sizes	31
0402	31
0603	32
0805.....	32
34. 3D Models vs Footprints.....	32
35. PCB Layer Structure	33
F.Cu	33
B.Cu	33
36. Ground Plane	33
37. Copper Zones	34
38. Ground Vias.....	34
39. Via Nets.....	35
40. Through-Hole Pads and Ground	36
41. Ratsnest Lines	36
42. Routing Philosophy	37
43. Trace Widths	37
Approximately 0.3 mm	37
Approximately 0.5 mm	37
44. Power Routing.....	38
45. Branching Power Nets	38
46. MCU Power-Pin Routing.....	38
47. Differential Pair Routing.....	39
48. Ground-Plane Continuity	39
49. PCB Placement.....	40
Decoupling capacitors.....	40
Crystal.....	40
USB connector	40

Voltage regulator.....	40
Communication headers	40
Mounting holes.....	40
50. PCB Grid and Snapping.....	40
51. Silkscreen	41
52. Fabrication Layer vs Silkscreen.....	42
53. Mounting Holes	42
54. Electrical Rules Check — ERC	43
No Connect marker	43
PWR_FLAG	43
55. Design Rules Check — DRC.....	43
56. Design Rules	44
57. Via Types.....	44
Through Via.....	44
Blind Via.....	45
Buried Via	45
Microvia	45
58. Board Stack-Up	45
59. PCB 3D Viewer.....	46
60. Bill of Materials	47
61. BOM Grouping.....	47
62. Manufacturer and MPN Fields.....	48
63. Manufacturing Files.....	49
Gerber Files	49
Drill Files	49
BOM.....	49
Position / Pick-and-Place File	49
64. Gerber Viewer	50
65. Final Manufacturing Workflow.....	51
66. Key Concepts Learned.....	52
67. Important Design Lessons	53
A schematic defines connectivity, not physical routing.....	53
Ratsnest lines are not routing instructions	53
Ground planes must actually be created	54

Vias are not automatically ground.....	54
Through-hole pads can connect directly to the ground plane	54
Decoupling placement matters	54
Footprint accuracy matters more than 3D appearance	54
DRC does not replace engineering understanding	54
68. Project Outcome	54
69. Project Reflection	55





1. Project Overview

This project involved designing a custom two-layer development board around the **STM32F103C8T6 ARM Cortex-M3 microcontroller** using KiCad.

The project was completed as a hands-on PCB design exercise based on **Phil's Lab's STM32 PCB tutorial**, with the goal of learning the complete workflow from schematic design to manufacturing files.

The board was designed to provide the main functionality expected from a small STM32 development platform, including:

- regulated 3.3 V power from USB
- STM32F103C8T6 microcontroller
- external 16 MHz crystal oscillator
- USB communication
- Serial Wire Debug programming/debug interface
- USART communication header
- I²C communication header
- BOOT0 selection
- reset circuitry
- digital and analog power filtering
- status LED
- mounting holes
- two-layer PCB layout
- bottom-layer ground plane

The purpose of the project was not to create a new STM32 architecture from scratch, but to understand how a real microcontroller PCB is designed, routed, verified, and prepared for manufacturing.

2. Main Components

The main components used in the design were:

Microcontroller

STM32F103C8T6

- ARM Cortex-M3 processor
- LQFP-48 package
- operates from approximately 3.3 V

- contains GPIO, UART/USART, I²C, SPI, ADC, timers, USB and debugging hardware

Voltage Regulator

AMS1117-3.3

The AMS1117-3.3 converts the approximately **5 V USB VBUS supply into a regulated 3.3 V supply** for the STM32 and the rest of the board.

It is important to distinguish this from a buck converter.

The AMS1117 is a **linear voltage regulator**, not a switching buck converter. It reduces the voltage by dissipating the excess energy as heat.

USB Connector

A **Micro-USB Type-B connector** was used.

The selected footprint was based on the Würth Elektronik connector:

629105150521

USB provides:

- VBUS
- D+
- D-
- GND

The unused ID and shield pins were intentionally marked as not connected in this design.

External Crystal

A **16 MHz crystal** was connected to the STM32 HSE oscillator pins.

Communication Headers

Three 1×4 pin headers were added for:

- SWD
- USART
- I²C

The headers use standard **2.54 mm pin spacing**.

Other Components

The board also contains:

- ceramic capacitors
- resistors
- ferrite bead
- LED
- SPDT BOOT0 switch
- four M2 mounting holes
- ground vias
- copper zones

3. Basic Power System

USB supplies approximately:

VBUS = 5 V

The STM32 requires approximately:

3.3 V

Therefore, the power path is:

USB VBUS → AMS1117-3.3 → 3.3 V rail

The regulated 3.3 V rail then powers:

- STM32 VDD pins
- analog supply filtering circuit
- communication headers
- BOOT0 circuit
- pull-up resistors
- status LED circuitry

4. Voltage Regulation

The AMS1117-3.3 linear regulator receives USB VBUS at its input and produces approximately 3.3 V at its output.

The regulator circuit included large capacitors on both sides of the regulator.

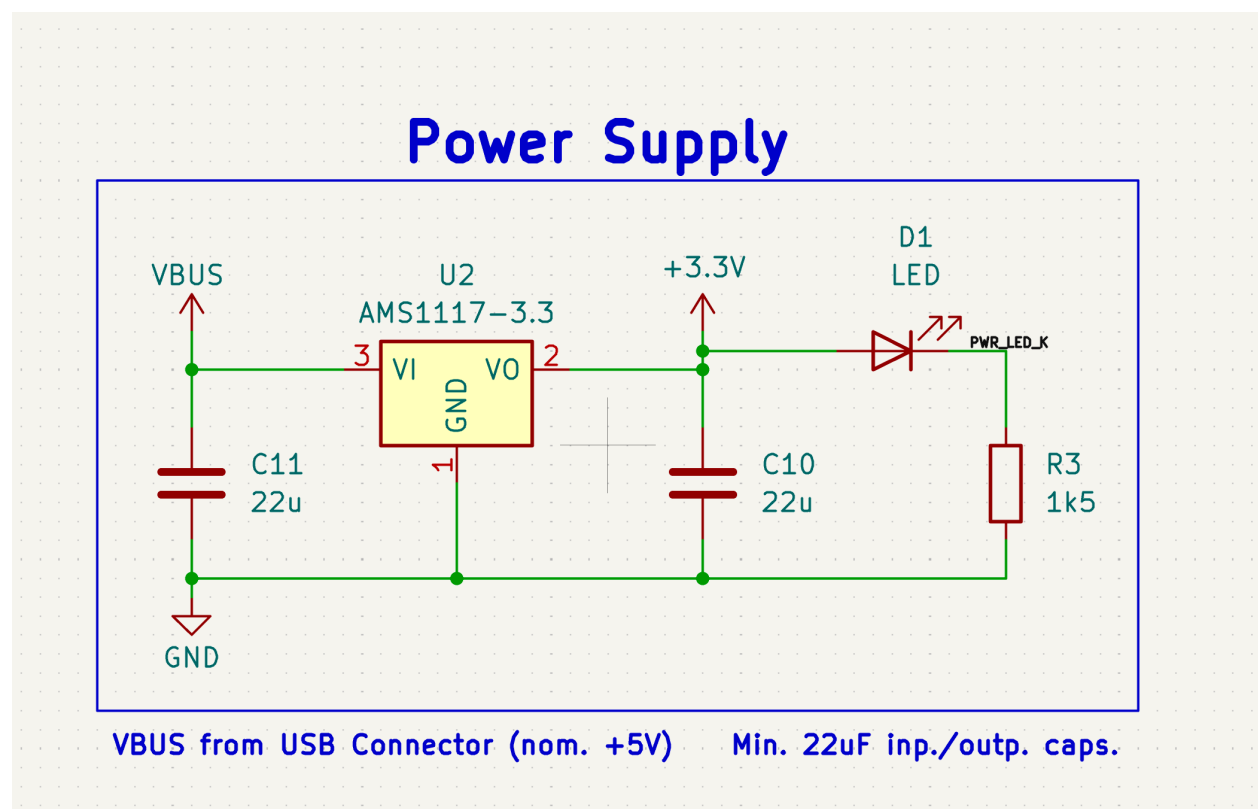
In this design:

- input capacitor: approximately **22 μ F**
- output capacitor: approximately **22 μ F**

These capacitors help stabilize the regulator and reduce voltage fluctuations.

A power LED and current-limiting resistor were also connected to the 3.3 V rail to provide a visual indication that the board is powered.

The LED resistor limits current through the LED so that excessive current does not damage it.



5. Decoupling Capacitors

One of the most important concepts learned during this project was decoupling.

Digital microcontrollers contain thousands of transistors switching extremely quickly.

When many internal circuits switch simultaneously, the MCU may suddenly require a small burst of current.

The power regulator cannot respond instantaneously because:

- it is physically farther away
- PCB traces have resistance
- PCB traces also have inductance

A decoupling capacitor placed very close to the MCU acts as a small local energy reservoir.

When the MCU suddenly requires current:

capacitor → supplies current immediately → MCU voltage remains stable

The main supply then replenishes the capacitor afterward.

5.1 Small Decoupling Capacitors

The STM32 has several VDD supply pins.

A 100 nF capacitor was placed for each VDD pin.

These small capacitors are especially effective at handling:

- very fast current changes
- high-frequency electrical noise

Because they deal with very fast signals, they should be placed physically very close to the MCU power pins.

Long PCB traces introduce inductance, which reduces their effectiveness.

Therefore:

small capacitor + short trace + close to VDD pin = effective high-frequency decoupling

6. Bulk Decoupling Capacitor

In addition to the 100 nF capacitors, a larger capacitor of approximately:

10 μ F

was placed near the MCU power rail.

The larger capacitor stores more energy than the smaller capacitors.

A useful simplified way of understanding the combination is:

100 nF capacitors $\rightarrow$ very fast disturbances

10 μ F capacitor $\rightarrow$ larger/slower power variations

Using both provides effective decoupling over a wider frequency range.

7. Coupling vs Decoupling Capacitors

Although coupling capacitors were not a major part of this particular board, the distinction is important.

Decoupling Capacitor

Connected between:

power $\rightarrow$ capacitor $\rightarrow$ ground

Its purpose is to stabilize a power rail and filter noise.

Coupling Capacitor

Normally placed in series with a signal:

signal $\rightarrow$ capacitor $\rightarrow$ next circuit

A coupling capacitor:

- blocks DC
- allows changing/AC signals through

Coupling capacitors are commonly used in:

- audio circuits
- amplifiers
- communication circuits

Therefore:

decoupling = power stabilization

coupling = signal transfer while blocking DC

8. Capacitor Behaviour During Power-Up

An important capacitor concept was also encountered in the reset circuit.

For steady DC, a capacitor eventually behaves approximately like an open circuit.

However, when initially uncharged, its behaviour is different.

At the instant voltage is applied:

uncharged capacitor → approximately short-like behaviour

As the capacitor charges, the current decreases.

Once fully charged:

capacitor → approximately open circuit for DC

This behaviour is useful in reset and filtering circuits.

9. Digital and Analog STM32 Power

The STM32 has separate supply pins including:

- VDD

- VDDA
- VSS
- VSSA

VDD

VDD powers the main digital circuitry inside the MCU.

VSS

VSS is digital ground.

VDDA

VDDA supplies the MCU's internal analog circuitry, including circuitry related to the ADC.

VSSA

VSSA is analog ground.

Both VDD and VDDA are still supplied with approximately **3.3 V DC**.

The word "analog" does not mean that VDDA is an AC or changing analog voltage.

It simply means that the supply is intended for the MCU's sensitive analog circuitry.

10. VDDA Filtering

Digital logic switching generates high-frequency electrical noise.

That noise could reduce the accuracy of sensitive analog circuits such as the ADC.

Therefore, a separate filtered 3.3 V analog rail was created.

The simplified structure is:

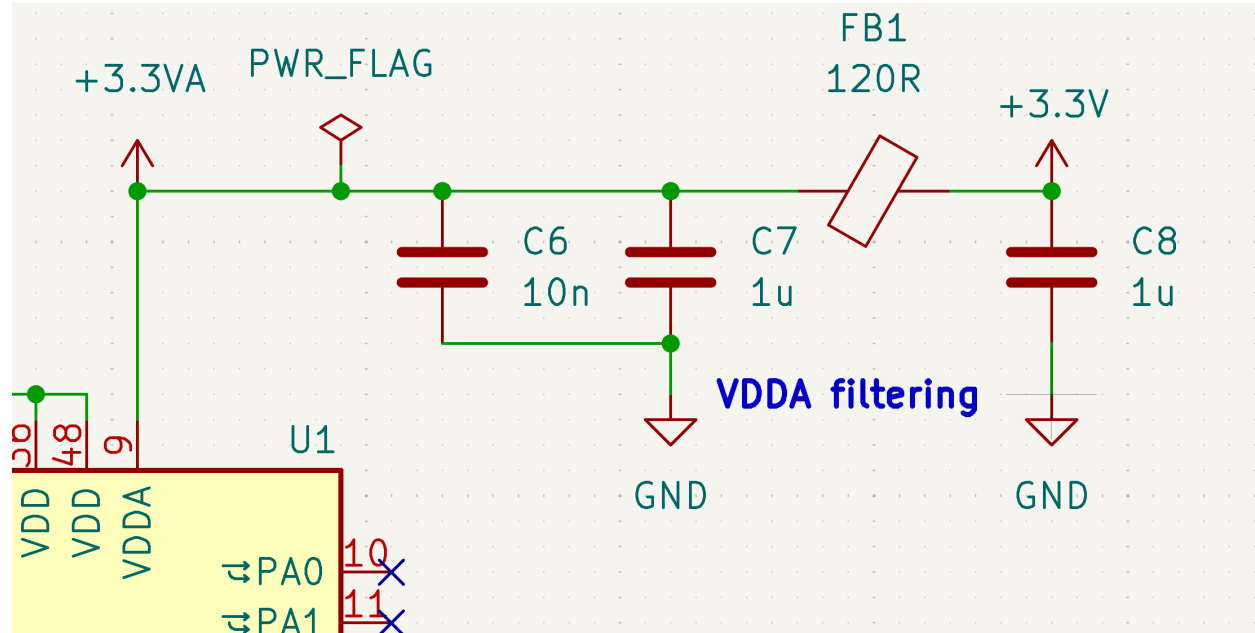
3.3 V → capacitor → ferrite bead → capacitors → 3.3VA → VDDA

The filtered rail was labelled:

+3.3VA

This is still a 3.3 V DC supply.

It is simply a cleaner version of the normal 3.3 V rail.



11. Ferrite Bead

A ferrite bead was placed between the normal 3.3 V supply and the analog supply.

The ferrite bead used in the schematic was labelled approximately:

120R

This value normally represents the bead's impedance at a specified high frequency rather than simply being a 120 Ω DC resistor.

A ferrite bead:

- allows normal DC current through with relatively low resistance
- presents greater impedance to high-frequency electrical noise

Therefore:

DC power passes → high-frequency noise is reduced

The capacitors around the ferrite bead help redirect the unwanted high-frequency noise to ground.

In this design the filtering network included approximately:

- 1 μ F before the ferrite bead
- 1 μ F after the ferrite bead
- 10 nF after the ferrite bead

Together, the bead and capacitors create a cleaner supply for VDDA.

12. PWR_FLAG

A KiCad **PWR_FLAG** was placed on the analog power section.

The PWR_FLAG is not a real physical PCB component.

It exists for KiCad's Electrical Rules Checker.

After components such as a ferrite bead, KiCad may not automatically understand that the resulting net is being actively powered.

The PWR_FLAG tells ERC:

"This net is intentionally powered."

This prevents false electrical-rule warnings.

13. Reset Circuit — NRST

The STM32 contains an active-low reset input called:

NRST

Its behaviour is:

NRST HIGH → MCU operates normally

NRST LOW → MCU resets

The STM32 provides an internal pull-up that normally keeps NRST high.

A capacitor of approximately:

100 nF

was connected between NRST and ground.

At power-up, the capacitor is initially uncharged.

This temporarily prevents NRST from instantly rising.

As the capacitor charges, NRST rises to its normal HIGH state.

The capacitor also helps filter short electrical disturbances that might otherwise accidentally reset the MCU.

Therefore, the NRST capacitor provides:

- cleaner power-up behaviour
 - increased immunity to reset-line noise
-

14. BOOT0

The STM32 BOOT0 pin determines where the MCU begins executing code after reset.

Two important states are:

BOOT0 LOW

The STM32 starts the user's application stored in internal Flash memory.

This is the normal operating mode.

BOOT0 HIGH

The STM32 starts the built-in ST bootloader stored inside the MCU's System Memory.

This bootloader can be used to load firmware using supported communication interfaces.

The important concept is that both the user's firmware and the built-in bootloader are stored inside the STM32.

The bootloader is not located on the connected computer.

15. BOOT0 Switch

An SPDT switch was used so BOOT0 could be manually connected to either:

- GND
- 3.3 V

Therefore, the user can select:

GND → normal Flash boot

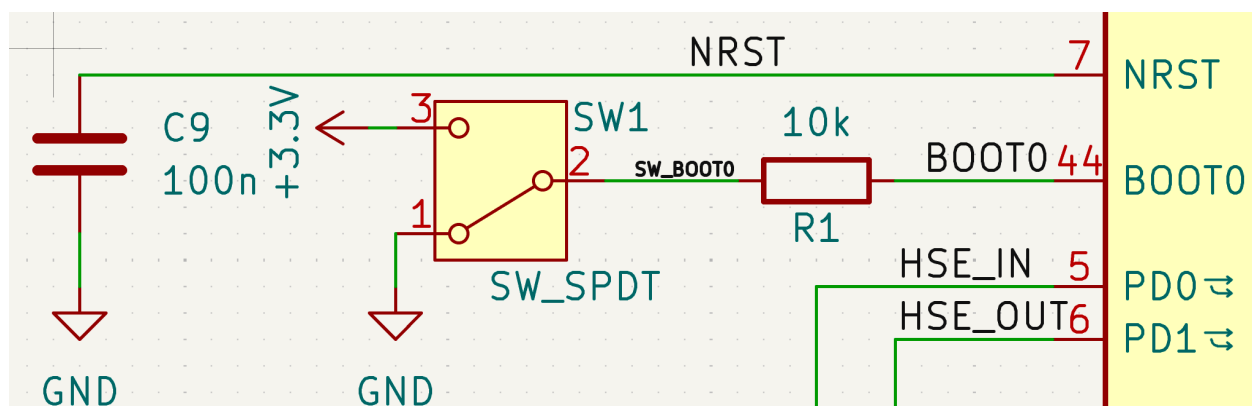
3.3 V → system bootloader

After changing BOOT0, the MCU must be reset for the new boot selection to take effect.

The tutorial circuit also placed a **10 kΩ resistor in series between the switch and BOOT0**.

Because BOOT0 is a high-impedance input, very little current normally flows through this resistor and therefore very little voltage is dropped across it.

In this specific wiring, the resistor is not acting as a conventional pull-down resistor.



16. External Crystal Oscillator

The STM32 can use an external high-speed oscillator for accurate timing.

A:

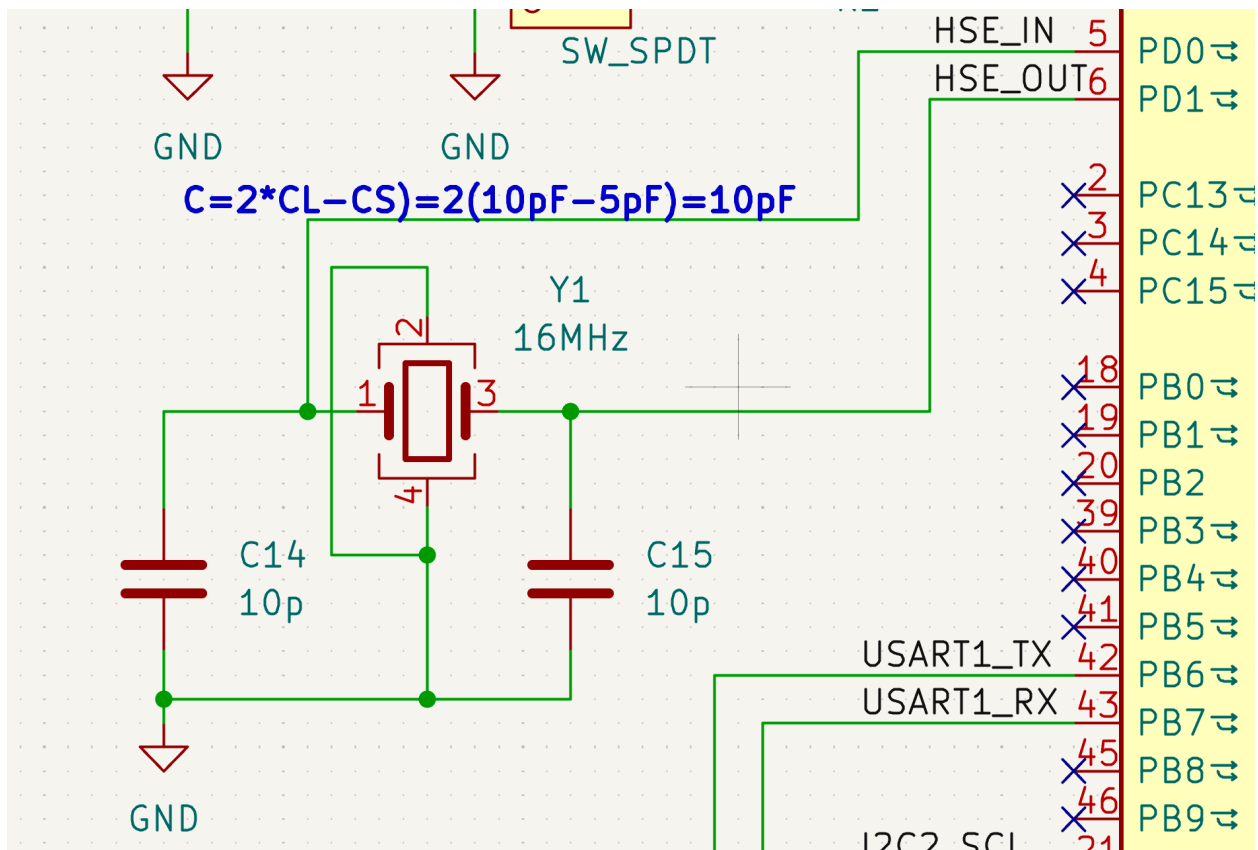
16 MHz crystal

was connected to the HSE pins:

- OSC_IN / HSE_IN
- OSC_OUT / HSE_OUT

The crystal is a passive resonating component.

The STM32 contains the active oscillator circuitry internally, while the external crystal determines the oscillation frequency.



17. Crystal Load Capacitors

Two capacitors were connected from the crystal terminals to ground.

The tutorial used the crystal load-capacitance relationship:

$$CL = (C1 \times C2) / (C1 + C2) + CS$$

where:

- CL = required crystal load capacitance

- C1 and C2 = external capacitors
- CS = stray/parasitic capacitance

For equal capacitors:

$$C1 = C2 = C$$

Using approximately:

- CL = 10 pF
- estimated stray capacitance = 5 pF

gives:

$$C \approx 10 \text{ pF}$$

Therefore, two approximately **10 pF capacitors** were used.

The 4-pad crystal footprint also has additional ground pads connected to GND.

18. USB Interface

A Micro-USB Type-B connector was connected to the STM32.

The main USB connections are:

- VBUS
- D+
- D-
- GND

The STM32F103 has built-in USB hardware.

The relevant MCU pins are:

PA11 → USB D-

PA12 → USB D+

These pins are fixed hardware capabilities of the STM32.

CubeMX/CubeIDE does not create these capabilities; they already exist inside the chip.

- with controlled spacing where practical

In KiCad, they were routed as a differential pair.

20. USB D+ Pull-Up Resistor

A **1.5 k Ω resistor** was connected from USB D+ to 3.3 V.

Its purpose is USB device detection.

When the board is powered, the resistor pulls D+ high.

If the USB cable is not connected, the computer cannot see this condition because there is no physical connection.

When the USB cable is connected, the host suddenly detects that D+ is being pulled high.

This indicates:

a USB Full-Speed device has been attached

The host then begins USB enumeration and communication.

After connection is detected, the USB hardware actively drives D+ and D− to transfer data.

21. USB ID Pin

The Micro-USB connector includes an ID pin.

This is primarily used for USB OTG functionality to determine whether a device should operate as:

- host
- peripheral

That function was not required in this design, so the pin was intentionally left unused.

22. USB Shield

The connector also has a shield connection associated with its metal outer shell.

The shield can be used for:

- electromagnetic shielding
- ESD handling
- chassis grounding strategies

In this tutorial design, it was intentionally left unconnected.

This should not be interpreted as a universal USB design rule; different products may connect the shield differently depending on EMC and enclosure requirements.

23. Serial Wire Debug

A Serial Wire Debug interface was included.

SWD is ARM's debugging/programming interface.

The two primary signals are:

SWDIO

Serial Wire Data Input/Output.

This is a bidirectional data line.

SWCLK

Serial Wire Clock.

This provides the clock used for SWD communication.

For the STM32F103:

PA13 → SWDIO

PA14 → SWCLK

These functions are built into the STM32 hardware.

24. What SWD Is Used For

An ST-Link programmer/debugger can be connected to the SWD header.

Using STM32CubeIDE, the developer can then:

- program firmware into Flash
- start and stop execution
- use breakpoints
- step through code
- inspect variables
- inspect registers
- debug the application

A typical SWD header includes:

- 3.3 V reference
- SWDIO
- SWCLK
- GND

The 3.3 V pin allows the debugger to know the target board's logic voltage.

25. USART / UART Interface

A separate header was provided for serial communication.

The STM32 peripheral is technically called:

USART

but it can operate in normal asynchronous UART-style communication.

The two most important signals are:

TX = Transmit.

RX = Receive.

When connecting two devices:

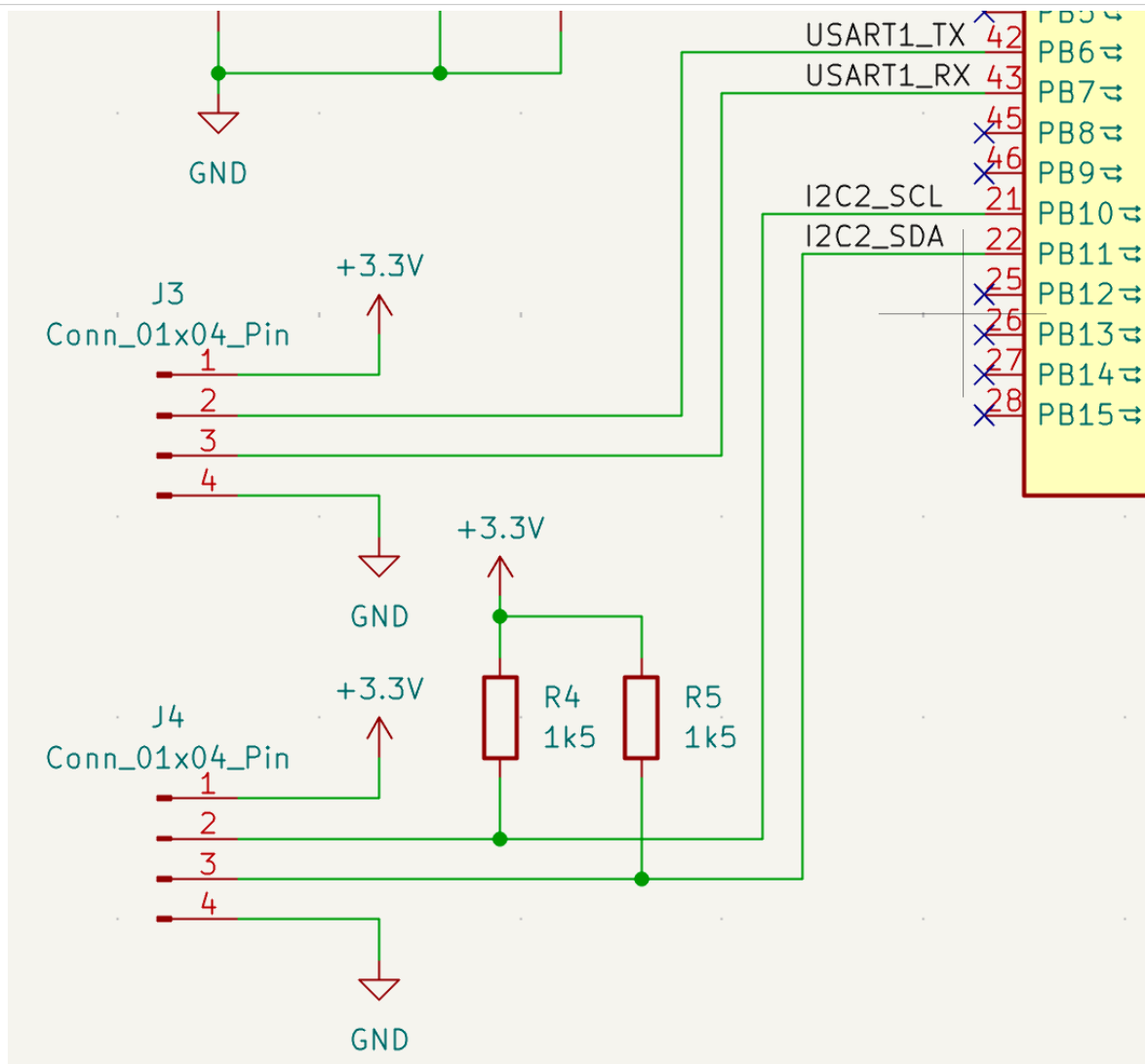
Device A TX → Device B RX

Device A RX ← Device B TX

UART does not normally require a separate clock signal.

A typical header therefore included:

- 3.3 V
- TX
- RX
- GND



26. I²C Interface

An I²C header was also provided.

I²C uses two signals:

SCL

Serial Clock.

SDA

Serial Data.

Multiple devices can share the same SCL and SDA wires.

Each device normally has an address, allowing the controller to communicate with a particular peripheral on the shared bus.

27. Why I²C Requires Pull-Up Resistors

I²C uses an **open-drain** signalling method.

An I²C device can:

- actively pull a line LOW
- release the line

It does not normally actively drive the line HIGH.

Therefore external resistors pull the lines HIGH whenever no device is pulling them LOW.

Two pull-up resistors are needed:

- one for SDA
- one for SCL

In this design, approximately:

1.5 k Ω

was used on each line.

Therefore:

device pulls LOW → logic 0

every device releases line → pull-up resistor creates logic HIGH

This also allows multiple devices to share the same communication wires without two devices directly fighting each other by driving opposite voltage levels.

28. Why UART Does Not Need These Pull-Ups

A normal UART TX output can actively drive:

- HIGH
- LOW

Therefore, it does not need an external resistor to create the HIGH state.

This is the basic difference:

I²C → open-drain → requires pull-ups

UART → push-pull output → normally does not require pull-ups

29. STM32CubeIDE / CubeMX

STM32CubeIDE and CubeMX were used to understand and configure the STM32's internal peripheral assignments.

They are separate from KiCad.

There is no automatic transfer of PCB wiring between CubeIDE and KiCad.

The workflow is:

1. Select the STM32 in CubeMX/CubeIDE.
2. Examine available peripheral functions.
3. Configure functions such as USB, UART, I²C and SWD.

4. Identify which physical MCU pins provide those functions.
5. Manually connect those pins correctly in KiCad.
6. After the physical board is manufactured, use CubeIDE and an ST-Link to program/debug the MCU.

One STM32 pin may support several alternate functions.

For example, depending on the pin, it may support:

- GPIO
- UART
- SPI
- I²C
- timer functions

CubeMX allows the designer to configure which supported function the firmware should use.

Therefore:

datasheet/pinout = what a pin is capable of

CubeMX/CubeIDE = what function is selected/configured

KiCad = how that physical pin is connected on the PCB

30. Schematic Capture in KiCad

After understanding the required circuits, the complete schematic was created in KiCad.

The main schematic blocks were:

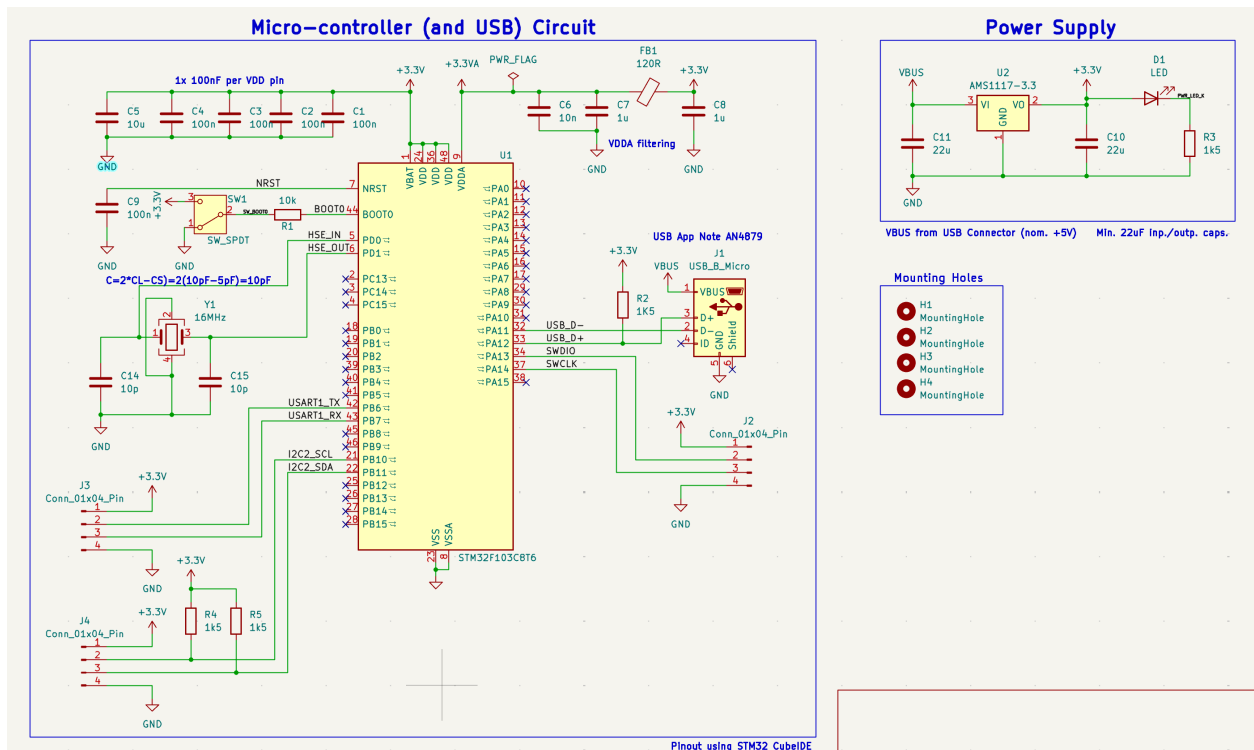
1. STM32 microcontroller
2. VDD decoupling
3. VDDA filtering
4. NRST circuit
5. BOOT0 circuit
6. external oscillator
7. USB
8. SWD
9. USART
10. I²C
11. power supply
12. status LED

13. mounting holes

Unused MCU pins were intentionally marked with **No Connect** symbols where appropriate.

This allows KiCad to distinguish between:

- accidentally unconnected pins
- intentionally unused pins



31. Symbols, Values and Footprints

An important KiCad concept learned during the project was the difference between:

Symbol

The schematic representation of a component.

Value

Its electrical identity or value.

Examples:

- 100 nF
- 10 k Ω
- STM32F103C8T6

Footprint

The physical copper pad pattern that appears on the PCB.

For example:

100 nF

does not determine the physical size of a capacitor.

A 100 nF capacitor might exist in many packages.

The footprint determines which physical package can actually be soldered to the board.

32. Selected Footprints

Representative footprints used included:

STM32

LQFP-48, 7 × 7 mm, 0.5 mm pitch

AMS1117

SOT-223

Small capacitors and resistors

Mostly:

0402 / 1005 metric

10 μ F capacitor

Approximately:

0603 / 1608 metric

22 μ F capacitors

Approximately:

0805 / 2012 metric

LED

0603

Ferrite bead

0603

Crystal

Approximately:

3.2 $\times$ 2.5 mm, 4-pin SMD

Headers

1 $\times$ 4, 2.54 mm pitch, vertical

Mounting holes

M2-compatible mounting-hole footprints.

33. Component Package Sizes

The project also introduced standard SMD package notation.

For example:

0402

Approximately:

1.0 $\times$ 0.5 mm

also called:

1005 metric

0603

Approximately:

1.6 × 0.8 mm

0805

Approximately:

2.0 × 1.25 mm

These dimensions matter when choosing real parts for manufacturing.

34. 3D Models vs Footprints

A footprint determines the actual physical manufacturing geometry:

- copper pads
- holes
- spacing
- mechanical mounting points

The 3D model is mainly a visual/mechanical representation.

A different or incorrect 3D model does not change the Gerber copper geometry.

For the BOOT0 switch, the original KiCad footprint did not include the desired 3D representation.

A separate 3D model was downloaded and associated with the footprint.

This later caused KiCad DRC to report that the footprint on the PCB did not exactly match the original copy in the KiCad library.

This warning was understood and was not an electrical routing error.

35. PCB Layer Structure

The board was designed as a:

2-layer PCB

The two copper layers are:

F.Cu

Front copper.

Used primarily for:

- signals
- power traces
- component connections

B.Cu

Back copper.

Used primarily as a continuous:

GND plane

If a board had four layers, KiCad could also provide internal copper layers such as:

- In1.Cu
- In2.Cu

36. Ground Plane

A large copper zone was created on B.Cu and assigned to:

GND

The zone covers almost the entire board.

When the zone is filled, KiCad connects the copper to:

- GND vias

- GND through-hole pads
- other GND copper

while automatically maintaining clearance from other nets.

This creates a low-impedance ground network across the board.

Benefits include:

- reduced ground resistance
- lower inductance
- easier routing
- improved return-current paths
- improved noise performance

37. Copper Zones

A copper zone is not automatically ground.

When creating the zone, the designer must explicitly select:

Net = GND

After drawing the zone, KiCad uses:

B — Refill Zones

to recalculate the copper fill.

KiCad automatically avoids:

- non-GND traces
- non-GND vias
- non-GND pads
- board-edge clearances

38. Ground Vias

A via is a plated hole that electrically connects copper layers.

A normal through-via consists of:

- a drilled hole
- copper plating on the inside wall
- copper pads around the hole

There is no separate wire inside the via.

The metal plating forms a hollow conductive cylinder.

A standard via used during the project was approximately:

0.7 mm via diameter / 0.3 mm drilled hole

The first value represents the outside copper diameter.

The second value represents the drilled hole diameter.

39. Via Nets

A physical via passing through a ground plane does not automatically become GND.

The via must belong to the:

GND net

If the via has no net, KiCad creates clearance around it.

Once the via is assigned GND:

F.Cu GND → via barrel → B.Cu ground plane

becomes a continuous electrical connection.

This prevents accidental shorts between unrelated nets and copper planes.

40. Through-Hole Pads and Ground

Through-hole connector pads already contain plated holes through the PCB.

Therefore, a through-hole GND pad can directly connect to the B.Cu ground plane.

It normally does not need a separate nearby via.

For example:

connector GND pin → plated through-hole → B.Cu GND

SMD pads are different.

An SMD GND pad located only on F.Cu needs:

- a GND via
- a front-side GND zone connected to vias
- or another physical ground path

to reach the B.Cu ground plane.

41. Ratsnest Lines

Before routing, KiCad shows thin connection lines called:

ratsnest / airwires

They represent required electrical connections from the schematic.

They are not physical PCB traces.

The designer does not need to follow their exact geometric path.

They simply indicate:

these two points still need to become electrically connected

The designer decides the best physical routing path.

42. Routing Philosophy

The goal is not simply to copy the exact trace shapes from a reference board.

Different PCB layouts can implement the same schematic correctly.

However, eliminating ratsnest lines alone is not enough.

Routing should also consider:

- current
 - noise
 - trace length
 - component placement
 - return paths
 - high-speed signals
 - manufacturability
-

43. Trace Widths

Two main trace widths were used during the project.

Approximately 0.3 mm

Used for:

- ordinary signals
- short connections around fine-pitch STM32 pads

Approximately 0.5 mm

Used for:

- 3.3 V power distribution
- larger-current supply routes

The MCU's small LQFP pads make very wide traces difficult directly at the pins.

Therefore a common strategy is:

wide main power trace → short narrower section into MCU pad

44. Power Routing

Power traces were intentionally wider than ordinary signal traces.

A wider trace provides:

- lower electrical resistance
- lower voltage drop
- better current capability

The exact physical route did not need to match the tutorial.

As long as the correct net was connected and sensible routing practices were followed, a different layout was acceptable.

45. Branching Power Nets

If several pads belong to the same 3.3 V net, they are electrically the same node.

A new 3.3 V branch can be taken from:

- another 3.3 V pad
- an existing 3.3 V trace
- an appropriate junction on the same net

The branch location is chosen mainly for:

- clean routing
 - short path
 - sensible current distribution
-

46. MCU Power-Pin Routing

Where two nearby STM32 3.3 V pads needed to be connected, a short approximately 0.3 mm trace could be used.

The ratsnest may appear directly between two nearby pins, but its exact geometric line is not a routing instruction.

The important requirements are:

- correct electrical connection
- sufficiently low impedance
- sensible physical routing

47. Differential Pair Routing

USB D+ and D− were routed together.

The pair should ideally maintain:

- similar trace lengths
- similar geometry
- consistent spacing
- minimal unnecessary detours

A predefined differential-pair width/gap can be configured in KiCad.

For highly controlled USB designs, exact differential impedance should be calculated using the PCB manufacturer's stack-up.

For this learning board, the routing was primarily intended to follow good USB differential-pair practice.

48. Ground-Plane Continuity

Because B.Cu was intended to be a ground plane, unnecessary signal traces on B.Cu were avoided.

Routing an unrelated signal such as 3.3 V across B.Cu would not automatically turn it into GND.

KiCad would maintain clearance between that trace and the GND zone.

However, the trace would cut through the ground plane and could interrupt return-current paths.

Therefore signals and power were kept mainly on F.Cu where practical.

49. PCB Placement

Component placement was performed before final routing.

Important placement decisions included:

Decoupling capacitors

Placed close to MCU VDD pins.

Crystal

Placed close to HSE pins.

USB connector

Placed near the board edge for physical access.

Voltage regulator

Placed near USB VBUS and the power-distribution section.

Communication headers

Placed near board edges for easy external connection.

Mounting holes

Placed near the corners.

Good placement simplifies routing and improves electrical performance.

50. PCB Grid and Snapping

KiCad uses a placement/routing grid.

The grid determines how far objects move between snap positions.

For example:

- 0.5 mm grid → coarse placement
- 0.25 mm / 0.1 mm grid → finer placement

A smaller grid is useful for:

- text placement
- fine-pitch routing
- aligning silkscreen

Snapping can also occur to nearby component anchor points.

51. Silkscreen

The silkscreen provides useful printed information on the physical PCB.

Instead of leaving generic reference labels such as:

- J2
- J3
- J4

visible on the final board, useful interface labels can be added such as:

- SWD
- USART
- I2C

The original reference designators can remain in the project internally or be moved to the fabrication layer.

Additional silkscreen elements can include:

- BOOT0 direction indication
- ON/OFF arrow
- polarity indicators
- connector labels
- project name
- logo

Text or graphics can also be placed on:

B.Silkscreen

for printing on the back of the PCB.

52. Fabrication Layer vs Silkscreen

The fabrication layer contains assembly/manufacturing drawing information.

Reference designators may be moved from F.Silkscreen to:

F.Fab

if they should remain in documentation but not be physically printed on the finished PCB.

Therefore:

F.Silkscreen → visible on physical front

B.Silkscreen → visible on physical back

F.Fab / B.Fab → fabrication documentation

53. Mounting Holes

Four mounting holes were included.

Their purpose is primarily mechanical.

They allow the PCB to be attached to:

- an enclosure
- standoffs
- another structure

The selected mounting holes do not perform an electrical function in this design.

They were excluded from the assembly BOM.

54. Electrical Rules Check — ERC

The Electrical Rules Checker operates mainly on the schematic.

ERC checks for issues such as:

- incorrectly driven power nets
- unintentionally unconnected pins
- conflicting pin types
- missing power sources
- other schematic electrical inconsistencies

Useful schematic tools included:

No Connect marker

Indicates that an unused pin is intentionally unconnected.

PWR_FLAG

Indicates to KiCad that a net is intentionally powered.

ERC does not prove that the circuit will function perfectly.

It checks whether the schematic violates the electrical rules that KiCad understands.

The designer must still understand the circuit.

55. Design Rules Check — DRC

After PCB routing, KiCad's Design Rules Checker was used.

DRC checks physical PCB-layout rules including:

- copper clearance
- track width
- via dimensions
- overlapping objects
- board-edge clearance
- unconnected nets
- footprint-related issues

The final board produced:

0 errors

0 unconnected items

One warning remained related to the BOOT0 switch footprint not matching the original KiCad library footprint.

This occurred because the local footprint had been modified, particularly when working with its 3D model.

Since the modification was intentional and the physical pad arrangement had been checked, the warning was understood rather than treated as a routing failure.

56. Design Rules

PCB manufacturers specify limits such as:

- minimum trace width
- minimum copper clearance
- minimum via diameter
- minimum drill size
- copper-to-edge clearance

KiCad's default values are generic and should not automatically be assumed to represent every PCB manufacturer.

For an actual production board, the PCB house's capabilities should be checked before finalizing the design rules.

57. Via Types

Several via types exist.

Through Via

Passes from front copper to back copper.

Used on this board.

Blind Via

Connects an outer layer to an internal layer.

Buried Via

Connects internal layers without reaching an outer surface.

Microvia

Very small laser-drilled via used primarily for high-density interconnect boards.

Because this was a simple two-layer PCB, normal through-vias were appropriate.

58. Board Stack-Up

The board used a standard two-layer FR-4 structure.

A typical structure is:

front copper

FR-4 dielectric

back copper

Common inexpensive PCB thickness is approximately:

1.6 mm

and standard copper thickness is often approximately:

35 μm / 1 oz

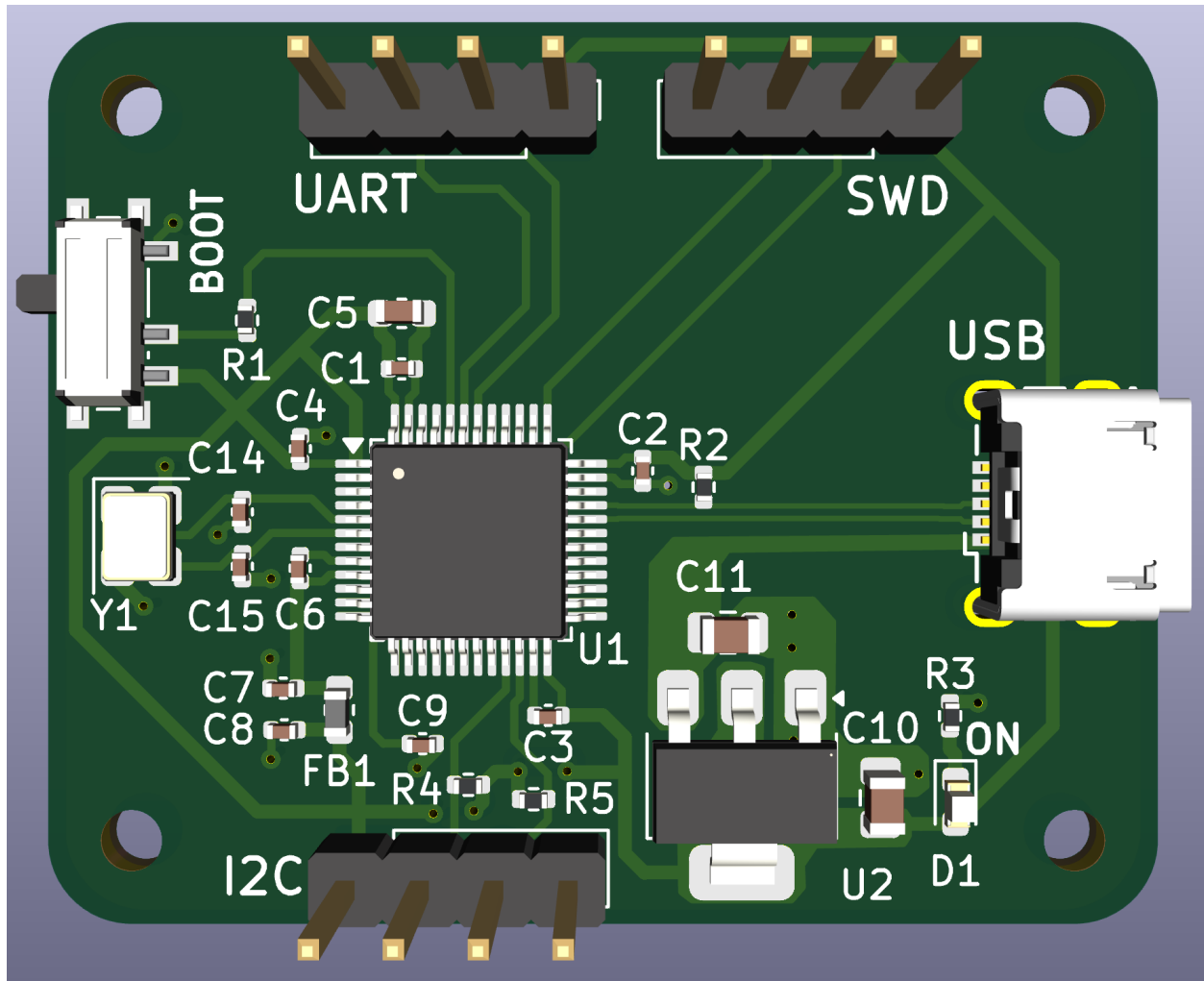
Exact values should ultimately be confirmed with the manufacturer.

59. PCB 3D Viewer

KiCad's 3D Viewer was used to inspect:

- component orientation
- physical appearance
- connector placement
- overall board layout
- silkscreen
- downloaded 3D models

The 3D Viewer is useful for mechanical inspection but does not replace checking the footprint against the component datasheet.



60. Bill of Materials

A Bill of Materials was generated from KiCad.

The BOM includes information such as:

- reference designator
- quantity
- value
- footprint
- datasheet

For actual assembly, additional fields should ideally include:

- manufacturer
- manufacturer part number / MPN

For example:

Manufacturer: STMicroelectronics

MPN: STM32F103C8T6

The exact MPN is important because simply specifying:

100 nF, 0402

does not uniquely identify a component.

Many manufacturers produce hundreds of capacitors matching that general description.

61. BOM Grouping

KiCad can group BOM rows using selected fields.

For example, grouping by:

- value
- footprint

means several identical 100 nF 0402 capacitors may appear as one BOM entry with a larger quantity.

For manufacturing, it is also useful to group using:

MPN

because components should only be grouped together if they are intended to use the exact same purchasable part.

Field	Column Label	Show Column	Group By
Reference	Reference	☒	☐
Value	Value	☒	☐
Footprint	Footprint	☒	☒
Datasheet	Datasheet	☒	☐
Description	Description	☐	☐
\$(QUANTITY)	Qty	☒	☐
\$(ITEM_NUMBER)	#	☐	☐
MPN	MPN	☒	☐
Manufacturer	Manufacturer	☒	☒
Sim.Pins	Sim.Pins	☒	☐
\$(DNP)	DNP	☒	☐
\$(EXCL...M_BOM)	Exclude from BOM	☒	☒
\$(EXCL...BOARD)	Exclude from Board	☒	☒

Q Filter

☐ Exclude DNP ☒ Show 'Exclude from BOM' ☒ Group symbols

Reference ^	Qty	Value	DNP	Exclude from B...	Exclude from B...	Footprint	
> C1-C4, C9	5	100n	☐	☐	☐	Capacitor_SMD:C_0402_1005Metric	~
C5	1	10u	☐	☐	☐	Capacitor_SMD:C_0603_1608Metric	~
C6	1	10n	☐	☐	☐	Capacitor_SMD:C_0402_1005Metric	~
> C7, C8	2	1u	☐	☐	☐	Capacitor_SMD:C_0402_1005Metric	~
> C10, C11	2	22u	☐	☐	☐	Capacitor_SMD:C_0805_2012Metric	~
> C14, C15	2	10p	☐	☐	☐	Capacitor_SMD:C_0402_1005Metric	~
D1	1	LED	☐	☐	☐	LED_SMD:LED_0603_1608Metric	~
FB1	1	120R	☐	☐	☐	Inductor_SMD:L_0603_1608Metric	~
> H1-H4	4	MountingHole	☐	☒	☐	MountingHole:MountingHole_2.2mm_M2	~
J1	1	USB_B_Micro	☐	☐	☐	Connector_USB:USB_Micro-B_Wuerth_629105	~
> J2-J4	3	Conn_01x04_Pin	☐	☐	☐	Connector_PinHeader_2.54mm:PinHeader_1x0	~
R1	1	10k	☐	☐	☐	Resistor_SMD:R_0402_1005Metric	~
R2	1	1K5	☐	☐	☐	Resistor_SMD:R_0402_1005Metric	~
> R3-R5	3	1k5	☐	☐	☐	Resistor_SMD:R_0402_1005Metric	~
SW1	1	SW_SPDT	☐	☐	☐	Button_Switch_SMD:SW_SPDT_PCM12	~
U1	1	STM32F103C8T6	☐	☐	☐	Package_QFP:LQFP-48_7x7mm_P0.5mm	https://www.st.com/resource
U2	1	AMS1117-3.3	☐	☐	☐	Package_TO_SOT_SMD:SOT-223-3_TabPin2	http://www.advanced-mon
Y1	1	16MHz	☐	☐	☐	Crystal:Crystal_SMD_3225-4Pin_3.2x2.5mm	~

62. Manufacturer and MPN Fields

Custom fields were added to the KiCad BOM configuration:

- Manufacturer
- MPN

These fields can initially remain blank while part selection is still in progress.

They can later be populated when exact components are selected.

For long-term project maintenance, it is preferable to store the final MPN information inside the KiCad project rather than only editing an external spreadsheet.

This ensures that regenerated BOMs continue to contain the correct part selections.

63. Manufacturing Files

After completing the PCB, manufacturing files were generated.

Important outputs include:

Gerber Files

Gerbers describe the physical PCB layers.

Typical Gerbers include:

- F.Cu
- B.Cu
- F.Mask
- B.Mask
- F.Silkscreen
- B.Silkscreen
- Edge.Cuts

If required for assembly:

- F.Paste
- B.Paste

may also be generated.

Drill Files

Drill files specify:

- through-hole component holes
- vias
- mounting holes

BOM

Specifies which components should be purchased and assembled.

Position / Pick-and-Place File

Specifies:

- component location
- rotation

- side of PCB

This file allows an automated assembly machine to position components.

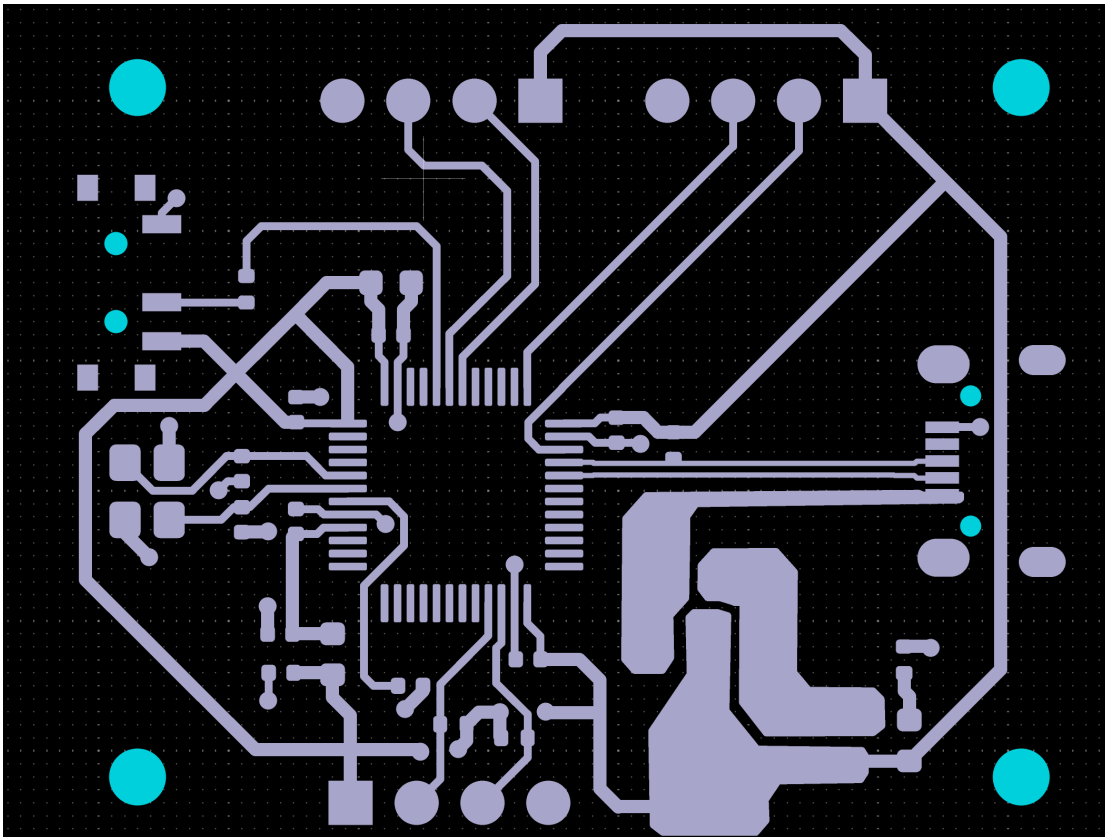
64. Gerber Viewer

The generated Gerber files can be opened using KiCad's Gerber Viewer before sending the design to a PCB manufacturer.

This allows verification of:

- copper
- solder mask
- silkscreen
- board outline
- layer alignment

Reviewing Gerbers is an important final step because Gerbers represent what the manufacturer will actually use to fabricate the PCB.



65. Final Manufacturing Workflow

The complete workflow learned in this project was:

Circuit understanding



Schematic capture



ERC



Footprint assignment



PCB board setup



Component placement



Signal and power routing



Ground vias



B.Cu ground plane



Silkscreen



DRC



3D inspection



BOM generation



Manufacturer/MPN selection



Gerbers



Drill files



Pick-and-place files



PCB fabrication / assembly

66. Key Concepts Learned

This project provided hands-on exposure to a large number of PCB-design concepts, including:

- STM32 microcontroller architecture
- MCU power pins
- digital vs analog power
- decoupling capacitors
- bulk capacitors
- ferrite-bead filtering
- NRST
- BOOT0
- external crystals

- oscillator load-capacitance calculations
- USB differential signalling
- USB device detection
- pull-up resistors
- UART
- I²C
- open-drain signalling
- SWD
- STM32CubeIDE/CubeMX
- schematic symbols
- footprints
- SMD packages
- datasheets
- two-layer PCB stack-up
- copper planes
- vias
- ground stitching
- ratsnest connections
- trace widths
- power routing
- differential-pair routing
- silkscreen
- ERC
- DRC
- BOM preparation
- manufacturer part numbers
- Gerbers
- pick-and-place files
- PCB manufacturing workflow

67. Important Design Lessons

Several practical lessons became especially clear during the design.

A schematic defines connectivity, not physical routing

The PCB does not need to visually match another board.

The schematic defines what must connect.

The PCB designer decides how those connections are physically routed.

Ratsnest lines are not routing instructions

They simply indicate uncompleted electrical connections.

Ground planes must actually be created

Setting B.Cu aside for ground does not automatically create a ground plane.

A GND copper zone must be explicitly created and filled.

Vias are not automatically ground

A via only connects to a GND plane if it belongs to the GND net.

Through-hole pads can connect directly to the ground plane

Their plated holes already pass through the PCB.

Decoupling placement matters

Small high-frequency capacitors must remain close to the MCU power pins.

Footprint accuracy matters more than 3D appearance

The footprint determines what gets manufactured.

DRC does not replace engineering understanding

A PCB can technically pass DRC and still contain a bad circuit design.

Electrical understanding, datasheet review and design-rule checking are all necessary.

68. Project Outcome

The completed design resulted in a fully routed two-layer STM32 development board containing:

- STM32F103C8T6
- 3.3 V regulated power
- filtered analog supply
- external 16 MHz oscillator
- USB interface
- SWD programming/debugging
- USART
- I²C

- BOOT0 control
- reset circuitry
- status LED
- mounting holes
- front-layer signal/power routing
- bottom-layer ground plane
- silkscreen labelling

The PCB passed KiCad's connectivity checks with:

0 unconnected items

and:

0 routing/design errors

apart from one understood warning caused by an intentionally modified switch footprint.

The final design was then prepared for manufacturing through generation of:

- BOM
- Gerbers
- drill files
- position files

69. Project Reflection

This was my first complete PCB-design project.

Rather than treating the tutorial simply as a set of steps to reproduce, I used the project to understand why each major circuit and PCB-design decision was being made.

By the end of the project, I was able to explain:

- why each STM32 power pin requires decoupling
- why bulk and small capacitors are used together
- how the VDDA ferrite-bead filter works
- how NRST and BOOT0 operate
- how the STM32 external oscillator works
- how crystal capacitors are selected
- why USB uses differential signalling
- why D+ requires a pull-up
- how SWD programming works

- how UART and I²C differ
- why I²C requires pull-up resistors
- how STM32CubeIDE relates to PCB design
- how footprints correspond to physical components
- how ground planes and vias work
- how PCB routing differs from schematic connectivity
- how ERC and DRC are used
- how a BOM and manufacturing files are prepared

The project therefore provided a practical introduction to the complete process of turning a microcontroller circuit into a manufacturable PCB.

Project attribution: This board was built as a hands-on PCB design exercise based on Phil's Lab's STM32 PCB tutorial, with the goal of learning the complete schematic-to-manufacturing workflow.